Topology-Aware Mapping for Photonic Interconnects in TPU-Class LLM Accelerators

Pranav Sitaraman
MIT 6.5930
sitaraman.pranav@gmail.com

Simone Surdi
MIT 6.5930
simonesurdi@gmail.com

***Abstract*—This project evaluates Lightmatter-style photonic interconnect as an emerging technology for TPU-class accelerator clusters. The question is not only whether optical links have lower energy per bit, but whether their distance-insensitive transmission, hardware multicast, and high-radix switching change the memory placements and distributed schedules that a TPU pod should use for LLM training and inference. We integrate a topology model with AccelForge by adding a logical cluster-memory level, extracting mapper traffic, pricing that traffic with a distance-aware network model, and feeding the network cost back into the memory hierarchy until the result stabilizes. The final study is intentionally narrow: 16-node AccelForge validation, a 64-chip TPU cube, compiler-visible sharding and cache-fill choices, and a 9216-chip Ironwood/Gemma-Gemini proxy model. The main result is that the inner AccelForge mapping is stable under the interconnect swap, but the distributed schedule above it changes. This distinction explains both the large inference gains and the more modest training gains.**

I. Introduction

Google's TPU pods are one of the clearest examples of modern accelerator design being a data-movement problem. A TPU7x/Ironwood pod has 9216 chips, 192 GiB of HBM per chip, 1200 GB/s bidirectional ICI per chip, and is advertised for both dense and MoE workloads [1]. The useful question for a hardware project is therefore not "can we make a faster wire?" but "which schedules become good when data movement changes?"

Lightmatter's Passage family is a concrete photonic reference point. The public M1000 EVK page reports 114.6 Tb/s aggregate bidirectional bandwidth and 2.3 pJ/bit system efficiency including laser power [3]; EVK100 reports a 3.2 Tb/s per-fiber class point [4], [5]. Photonics is appealing at pod scale because the energy is mostly conversion and SerDes energy, while transmission over fiber or waveguides is nearly distance independent. Electrical links have low conversion overhead, but pay transmission energy as distance and topology grow.

Our thesis is that photonic interconnect is not a constant-factor link substitution. It is useful when it lets the system use a different *distributed schedule*: multicast cache fill instead of repeated unicasts, pooled-HBM streaming instead of local-only caching, and flatter topologies instead of hop-heavy meshes or tori. We contribute:

- an AccelForge-integrated logical-memory feedback loop for modeling network cost in a memory hierarchy;
- a distance-aware electrical/chiplet/photonic link model;
- a clear separation between AccelForge's inner mapping and the XLA/GSPMD-like distributed schedule above it;
- a compact end-to-end evaluation from 16 nodes to a 9216-chip Ironwood-class superpod.

II. Related Work and Class Context

Mapping and dataflow. In the course terminology, an einsum defines the computation but not the order in which it runs. A mapping chooses partitioning, temporal loop order, spatial loop order, tile sizes, and which tensors are resident at each memory level. The Turbo-Charged Mapper paper makes this precise by separating dataflow, tile shape, and dataplacement; it also argues that hardware comparisons are unreliable if the mapper is not allowed to find a good mapping [7]. That warning shapes our method. We do not report an interconnect result directly from a spreadsheet. We run AccelForge, observe the traffic implied by the mapping, charge the traffic through the topology, then rerun the mapper with the updated logical-memory cost.

Photonic accelerator modeling. Prior photonic DNN modeling has mostly focused inside the accelerator. CiMLoop for Photonics, for example, models photonic compute-in-memory components across electrical, analog, optical, and digital domains [8]. Our project asks a different question: if each TPU-like chip is held fixed, how does the *inter-chip* fabric change the memory hierarchy and distributed schedule?

TPU and compiler stack. TPU v5p and TPU7x both expose high-bandwidth ICI as a central system feature [1], [2]. XLA/GSPMD and PyTorch/XLA SPMD provide the software analogy for this project: users describe sharding over a device mesh, and the compiler inserts collectives based on those sharding choices [6]. We do not implement an XLA pass. Instead, we model the relevant choices explicitly: sharded collectives, remote pooled-HBM streaming, replicated HBM cache, and photonic multicast cache fill.

III. Model and Experiment Spine

Figure 1 shows the final experiment spine. The model has four layers:

1. Workload. Each LLM stage is represented as a GEMM-style einsum using `single_matmul.yaml`. The six AccelForge proxies cover decode MLP, KV attention, prefill projection, long-context prefill, MoE expert dispatch, and training weight gradients.

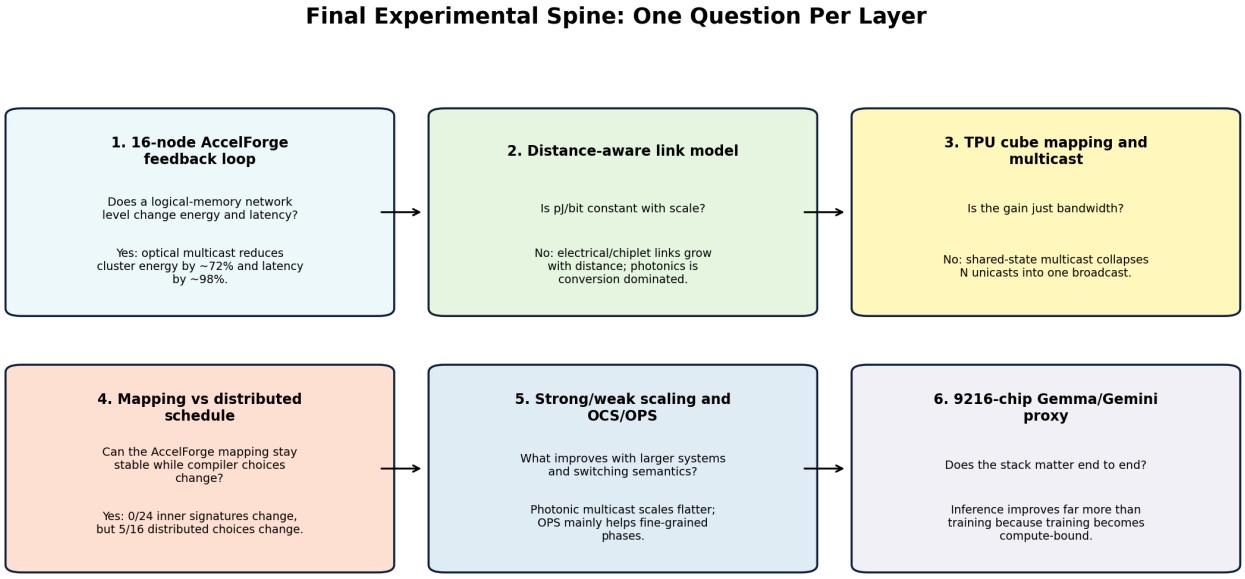


Fig. 1: The final project is organized as one evidence chain. Each experiment answers the question left by the previous one.

2. Architecture. The AccelForge architecture adds `LogicalClusterMemory` above per-chip `HBM`, `UnifiedBuffer`, registers, and MACs. This logical level represents the union of accelerator memories in a multi-chip system.

3. Inner mapping. AccelForge emits the mapping: storage, temporal, spatial, and compute nodes with tile sizes. We fingerprint this as a compact signature so we can ask whether the mapper actually changed.

4. Distributed schedule. Above the inner mapping, a `COMM_PATTERN` and compiler-strategy model decide whether tensor traffic is node-to-node, node-to-DRAM, or shared-DRAM multicast. This is where the XLA/GSPMD analogy lives. The key clarification is that layer 3 can remain unchanged while layer 4 changes.

For traffic of B bits, average hop count h, average per-hop distance $\ell(N)$, link bandwidth BW, contention $C(N)$, conversion energy e_c, per-mm transmission energy e_{tx}, router delay t_r, propagation delay t_p, and switching delay t_s, we use

$$E_{\text{net}} = B(e_c + h\,\ell(N)e_{tx}),$$

$$T_{\text{net}} = \frac{B}{BW/C(N)} + ht_r + h\,\ell(N)t_p + t_s.$$

Electrical and chiplet links are distance-sensitive; photonic links are conversion dominated. Multicast-capable optical links pay the unique payload once, while electrical and chiplet baselines pay repeated unicasts.

TABLE I: Compact assumptions used in the final submission.

Item	Value	Why it matters
Ironwood pod	9216 chips; 192 GiB HBM/chip; 1200 GB/s ICI/chip	Full-system scale
M1000	57.3 Tb/s one-way; 2.3 pJ/bit incl. laser	Main photonic fabric
Electrical	0.2 pJ/bit conversion + 1.96 pJ/bit/mm	Distance-sensitive baseline
Photonic	2.3 pJ/bit conversion + 5×10^{-5} pJ/bit/mm	Nearly flat with distance
Chiplet	3.5 Tb/s; 6 pJ/bit per hop	Stronger non-optical baseline
Mapper loop	traffic → topology cost → rerun	AccelForge-grounded method

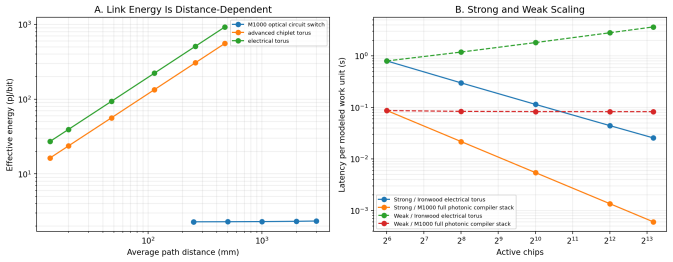


Fig. 2: Electrical and chiplet links become more expensive as average path distance grows; photonic links are nearly flat. The same model separates strong scaling from weak scaling.

IV. Results

A. Distance and Scaling

Figure 2 is the physical reason for the rest of the project. At short distance, electrical links can be competitive because they avoid optical conversion. At pod-scale distances, their distance-dependent term dominates. This also explains why the project keeps strong and weak scaling separate. Strong scaling asks whether a fixed work unit becomes faster with more chips; weak scaling asks whether aggregate work can grow while per-chip work stays similar. Photonics helps both when traffic contains shared state or multicast; it helps less

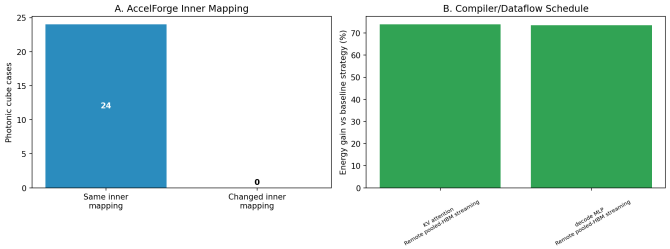


Fig. 3: The AccelForge inner mapping is stable, but the compiler-visible distributed schedule changes. This resolves the apparent contradiction in the results.

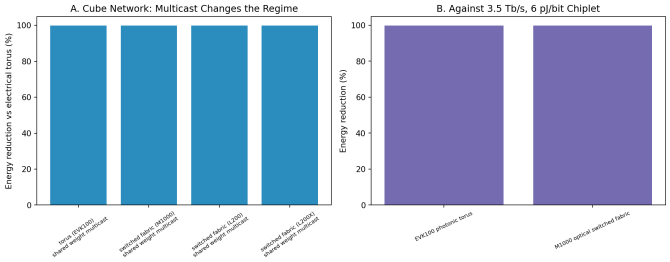


Fig. 4: The largest cube-level gains come from changing communication semantics: shared-state multicast replaces repeated unicasts.

when traffic is purely local or compute dominates.

B. Mapping Versus Distributed Schedule

Figure 3 is the most important modeling result. Across the photonic TPU-cube cases, 0 of 24 inner AccelForge signatures changed. That does not mean the system behavior is unchanged. It means the local GEMM dataflow and tile residency remain optimal after the logical-memory feedback loop. The distributed schedule above that mapping changes: 5 of 16 workload–architecture choices move from node-to-node exchange to shared-DRAM multicast, and the compiler-strategy model selects remote pooled-HBM streaming or photonic multicast cache fill when those policies become cheap. The best changed compiler strategy reduces energy by 73.86% relative to the baseline schedule.

C. Multicast, Not Just Bandwidth

The advanced chiplet baseline is deliberately strong: 3.5 Tb/s and 6 pJ/bit per hop. It prevents the project from claiming that photonics wins only because we compared against a weak electrical link. The result in Fig. 4 is that M1000 and EVK100 still win on shared-state multicast because topology semantics changed. The best pure TPU-cube network case reaches 99.93% energy reduction, and the AccelForge mapping-aware cube study reaches 77.45% energy reduction. On node-to-node traffic the gain is much smaller, which is exactly the expected negative result: if the schedule cannot use multicast or flatten the topology, photonics is mostly a link improvement.

D. End-to-End Superpod

The final model uses public Gemma 4 open-model anchors (26B MoE and 31B dense) and Gemini-class stress proxies.

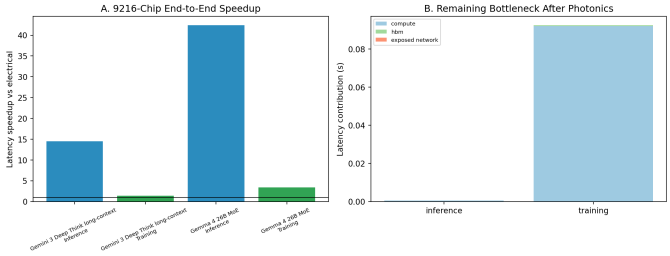


Fig. 5: End-to-end results on a 9216-chip Ironwood-class pod. Inference benefits far more than training because training becomes compute dominated after communication is reduced.

TABLE II: Final headline results.

Layer	Result	Interpretation
16-node AccelForge	71.95% energy, 97.57% latency reduction	Logical network cost matters
TPU cube network	99.93% energy reduction	Multicast collapses repeated traffic
TPU cube mapping	0/24 inner mappings changed	Local dataflow is stable
Distributed schedule	5/16 choices changed	Compiler-visible mapping changes
Compiler strategy	73.86% best energy gain	Remote HBM/multicast can be optimal
Superpod inference	42.43× Gemma 4 MoE speedup	Decode/cache/expert routing benefits most
Superpod training	3.37× Gemma 4 MoE speedup	Communication improves, then compute dominates

Gemini parameter counts are not public, so those workloads are explicitly treated as proxies. In the full M1000 photonic compiler stack, Gemma 4 26B MoE inference reaches 42.43× modeled latency speedup and 94.05% energy reduction. The larger long-context Gemini stress proxy reaches 14.50× inference speedup. Training improves less: 3.37× for the Gemma 4 MoE proxy and 1.43× for the larger stress proxy, because the dominant term becomes matrix-multiply compute rather than exposed network stall.

V. Discussion and Limitations

The strongest project claim is conditional: photonics is most valuable when the system exposes multicast, pooled memory, or schedule choices to the compiler. It is not a universal accelerator multiplier. Workloads with little inter-chip traffic, short electrical paths, or compute-dominated training phases improve less. This is why the paper reports negative results (unchanged inner mappings and modest training gains) alongside the positive multicast and inference results.

The model includes AccelForge action counts, HBM costs, distance-dependent link energy, hop count, contention, multicast volume, OCS/OPS sensitivity, and a stage-level overlap factor. It does not include packet-level queueing, thermal drift, laser aging, detailed area/cost, exact Google production XLA code generation, or model-quality effects. These are future-work items, not hidden assumptions.

The project is reproducible with the repository targets `make study`, `make notebook`, `make test`, and `make deliverables`. The final presentation folder contains only six submission figures

and two compact tables; the parent `results/` directory contains the raw CSV audit trail.

VI. LLM USAGE

Software assistance was used for code navigation, drafting, and editing. The quantitative results, figures, tests, and PDFs are generated from the checked-in scripts and were validated with the regression suite. The main places where assistance was least useful were numerical judgment and scope: we had to make the final claims narrower than the raw experiment space and state assumptions explicitly.

REFERENCES

[1] Google Cloud, "TPU7x (Ironwood)," 2026. [Online]. Available: https://docs.cloud.google.com/tpu/docs/tpu7x

[2] Google Cloud, "Cloud TPU v5p," 2026. [Online]. Available: https://cloud.google.com/tpu/docs/v5p

[3] Lightmatter, "Passage M1000 EVK," 2026. [Online]. Available: https://lightmatter.co/products/passage-m1000-evk/

[4] Lightmatter, "Passage EVK100," 2026. [Online]. Available: https://lightmatter.co/products/passage-evk100/

[5] Lightmatter, "Passage," 2026. [Online]. Available: https://lightmatter.co/products/passage/

[6] PyTorch/XLA, "SPMD User Guide," 2026. [Online]. Available: https://docs.pytorch.org/xla/master/spmd.html

[7] M. Gilbert, T. Andrulis, V. Sze, and J. Emer, "The Turbo-Charged Mapper: Fast and Optimal Mapping for Accelerator Modeling and Evaluation," arXiv:2602.15172, 2026.

[8] S. Zheng *et al.*, "CiMLoop for Photonics," MIT 6.5930 advanced technology material, 2026.

[9] Google, "Gemma 4," 2026. [Online]. Available: https://blog.google/innovation-and-ai/technology/developers-tools/gemma-4/